Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

TRAFFIC SIGN RECOGNITION WITH HOG–COLOUR FEATURES AND AN SVM CLASSIFIER

Submitted in partial fulfilment of the requirements in the subject of

Pattern Recognition and Applications (PRA) Honours Mini Project (DJS23ECH1801)

by

SHRUTIK ACHARYA (E149)

SIDDHAANT PANDIT (E151)

Electronics & Telecommunication Engineering

Dwarkadas J. Sanghvi College of Engineering

2026 – 2027

Table of Contents

CHAPTER NO.	SECTION NO.	TOPIC	PAGE NO.
		Certificate	i
		Abstract	1
1.		**Introduction**	**1**
	1.1	Problem Statement	1
	1.2	Overview	1
	1.3	Objectives	1
	1.4	Literature Review	1
2.		**Methodology**	**2**
	2.1	System Overview	2
	2.2	Overall Workflow	3
	2.3	System Architecture Diagram	3
	2.4	System Components	3
	2.5	Data Pre-Processing	4
	2.6	Model Implementation	4
3.		**Results and Discussion**	**5**
	3.1	Classification Accuracy on the Test Set	5
	3.2	Per-Class Performance of the SVM	5
	3.3	Detection Results	6
	3.4	Discussion	7
	3.5	Conclusion	7
	3.6	Future Scope	7
		References	**8**
A		**Appendix A: Python Code**	**9**

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)
Department of Electronics & Telecommunication Engineering

Certificate

This is to certify that the project entitled **"TRAFFIC SIGN RECOGNITION WITH HOG–COLOUR FEATURES AND AN SVM CLASSIFIER"** is a bonafide work of

1. SHRUTIK ACHARYA (Roll No. E149**)**

2. SIDDHAANT PANDIT (Roll No. E151**)**

submitted in partial fulfilment of the requirements in the subject of **Pattern Recognition and Applications (PRA) Honours Mini Project (DJS23ECH1801)** in Electronics & Telecommunication Engineering.

Teacher-in-charge
Dr. Vishakha Kelkar

Head of Department
Dr. Amit Deshmukh

Abstract

Traffic Sign Recognition (TSR) must read 43 kinds of road sign from small, blurry images and also locate the signs inside a full road scene. This project builds a classical TSR pipeline on the GTSRB benchmark: signs are located by HSV colour segmentation and contour filters, described by HOG shape and HSV colour features, reduced with PCA and classified into 43 sign classes plus a background class by an RBF-SVM, which is compared with k-NN. On 12,630 unseen test images the SVM reaches 87.2% accuracy (macro F1 0.848) against 68.7% for k-NN, and a real Speed limit 20 photograph is detected at 99% confidence. The implementation uses Python, OpenCV and scikit-learn in Google Colab.

1 Introduction

1.1 Problem Statement

Road signs tell drivers and driver-assistance systems what to obey. A TSR system must read 43 kinds of sign from small, blurry images taken in sun, shade or rain, and must also find the signs inside a full road scene.

1.2 Overview

Modern TSR systems use deep networks, which need GPUs and large labelled datasets and are harder to explain. Hand-crafted features with a support vector machine train in minutes on a CPU, can be inspected step by step, and give a clear baseline. This project builds an end-to-end system on GTSRB: colour segmentation locates signs, HOG and HSV features describe them, PCA reduces them, and k-NN and an RBF-SVM are compared on 12,630 unseen test images.

1.3 Objectives

The main objective is to build a fast and explainable system that detects and recognises traffic signs. Specific objectives include:

- To build a training set from GTSRB with augmented copies and a background class
- To pre-process crops (32×32, denoise, CLAHE) and extract a 356-value HOG–colour vector
- To reduce features with PCA and compare k-NN with an RBF-SVM by 3-fold grid search
- To locate signs in full scenes using HSV masks, a 0.60 threshold and non-max suppression

- To evaluate on all 12,630 test images with accuracy, a per-class report and a confusion matrix

1.4 Literature Review

Table 1 summarises the benchmark paper [1] and five recent papers [2]–[6] (2024–2025) that cover the GTSRB dataset, TSR surveys, YOLO-based detectors and deep classifiers.

Ref.	Title and authors	Novel contribution	Year and venue
[1]	**Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition** *J. Stallkamp, M. Schlipsing, J. Salmen, C. Igel*	Introduces the GTSRB dataset (50,000+ images, 43 classes), reports the IJCNN 2011 competition results and compares algorithms with human performance; CNNs scored highest.	2012, Neural Networks, Vol. 32, pp. 323–332
[2]	**A Comprehensive Survey and Analysis of Traffic Sign Recognition Systems With Hardware Implementation** *N. Triki, M. Karray, M. Ksantini*	Surveys TSR detection and classification (colour, shape, machine learning, deep learning) and tests TSR on Raspberry Pi and Jetson Nano.	2024, IEEE Access, Vol. 12, pp. 144069–144081
[3]	**Small traffic sign recognition method based on improved YOLOv7** *B. Meng, W. Shi*	Improves YOLOv7 for small signs in complex scenes with SPPFCSPC features, an S-CARAFE up-sampler and a Normalized Wasserstein Distance metric.	2025, Scientific Reports, Vol. 15
[4]	**YOLO-BS: a traffic sign detection algorithm based on YOLOv8** *H. Zhang, M. Liang, Y. Wang*	Adds a small-object detection layer and a BiFPN to YOLOv8, reaching 90.1% mAP50 at 78 FPS on TT100K.	2025, Scientific Reports, Vol. 15
[5]	**Efficient traffic sign recognition using YOLO for intelligent transport systems** *C. Wang, B. Zheng, C. Li*	Tunes YOLOv5 with k-means++ anchors and compares s/m/x variants, reaching 98.1% mAP at 45 FPS on CCTSDB.	2025, Scientific Reports, Vol. 15
[6]	**Comparative Experiment and Performance Evaluation of Three Different Models of Traffic Sign Recognition System** *L. Wang*	Compares CNN, MobileNet and ResNet on GTSRB: ResNet 99.70%, CNN 98.27% and MobileNet 75.40% accuracy.	2025, ITM Web of Conferences, Vol. 78 (CSEIT 2025), 03011

Table 1: Summary of the literature reviewed

Deep networks are the most accurate but need GPUs and large labelled sets and are harder to explain. Classical pipelines, such as the 87.2% HOG–colour SVM here, train in minutes on a CPU, can be inspected stage by stage and give a clear baseline, with learned detectors as the natural next step.

2 Methodology

2.1 System Overview

The system is a modular supervised pipeline that turns a road scene into labelled signs. Labelled GTSRB crops train an RBF-SVM on HOG shape and HSV colour features, and each stage (data preparation, pre-processing, feature extraction, training and detection) can be inspected or replaced, for example by a learned detector or a small CNN.

2.2 Overall Workflow

GTSRB train and test splits are loaded and up to 200 images per class are kept for training. The crops are augmented, 1,800 background patches are added, and each crop is resized to 32×32, blurred and contrast-equalised with CLAHE. HOG (324 values) and HSV histograms (32 values) form a 356-value vector, which passes through StandardScaler and PCA (95% variance). k-NN and RBF-SVM are compared by 3-fold grid search and the best SVM is refitted with probabilities. For a full scene, a colour mask finds candidate boxes, the SVM scores each, boxes with at least 0.60 are kept and non-max suppression removes duplicates.

2.3 System Architecture Diagram

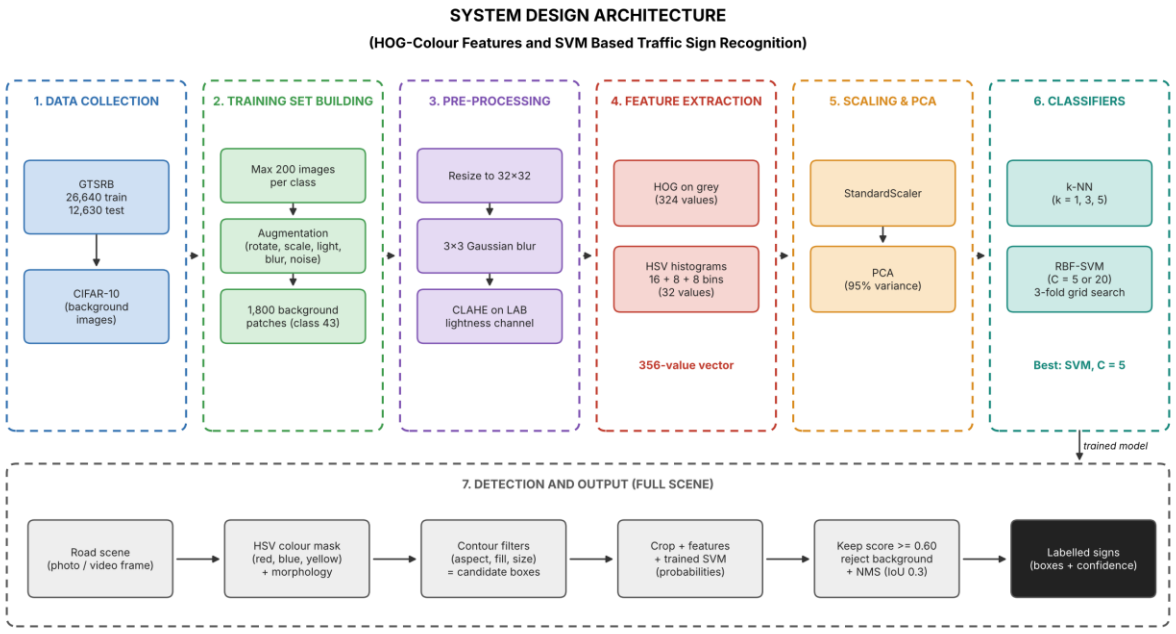


Figure 1: System architecture of the HOG–colour SVM traffic sign recognition pipeline

2.4 System Components

- **Data Input:** GTSRB (26,640 train, 12,630 test) and CIFAR-10 (background images only).
- **Training Set:** at most 200 images per class, one augmented copy each and 1,800 background patches (18,460 samples).
- **Pre-processing:** 32×32 resize, 3×3 Gaussian blur and CLAHE on the LAB lightness channel.
- **Feature Extraction:** HOG on grey (324 values) plus HSV histograms (32 values).
- **Classification:** StandardScaler, PCA and k-NN / RBF-SVM tuned by 3-fold grid search.
- **Detection:** HSV masks, contour checks, SVM score of at least 0.60 and non-max suppression at IoU 0.3.
- **Evaluation:** accuracy, per-class report and confusion matrix on 12,630 test images.

2.5 Data Pre-Processing

GTSRB (tanganke/gtsrb) gives 26,640 training and 12,630 test images; CIFAR-10 provides 50,000 images used only for background examples. At most 200 images per class are kept (8,330 sign crops), the test set is untouched, and 1,500 CIFAR-10 images plus 300 flat-colour patches form the background class (class 43).

```
8330 train | 12630 test
```

```
Total training samples: 18460
```

Figure 2: Dataset sizes after loading, sampling and building the training set

Each crop gets one augmented copy using rotation (±15°), scale (0.85–1.15), shift (up to 8%), brightness and contrast jitter, blur and noise, giving 18,460 training samples (8,330 + 8,330 + 1,800). Crops are then resized to 32×32, blurred (3×3) and contrast-equalised with CLAHE on the LAB lightness channel.



Figure 3: Original sign crops (top row) and their augmented copies (bottom row)

2.6 Model Implementation

Features: HOG on the grey image (9 bins, 8×8 cells, 16×16 blocks) gives 324 shape values and HSV histograms (16 + 8 + 8 bins) give 32 colour values, forming a 356-value vector.

```
(18460, 356) (12630, 356) 11.1s
```

Figure 4: Shape of the training and test feature matrices and the extraction time

Scaling, PCA and classifiers: the data is split 80/20 (stratified) and a pipeline applies StandardScaler, PCA (95% variance) and the classifier. The best k-NN is k = 5 with distance weights (validation 0.8567); the best RBF-SVM is C = 5 (validation 0.9228, grid search 146 s). The final SVM is refitted with probabilities (118 s) and saved as traffic_sign_svm.pkl.

```
k-NN best: {'clf__n_neighbors': 5, 'clf__weights': 'distance'} | val acc 0.8567
SVM best: {'clf__C': 5} | val acc 0.9228 | 146s

Trained in 118s
['traffic_sign_svm.pkl']
```

Figure 5: Grid search results for k-NN and SVM, and final SVM training time

Detection: HSV masks for red, blue and yellow are cleaned with a 5×5 ellipse; contours covering 0.04–25% of the image with aspect 0.6–1.5, fill above 0.30 and a side of 16 px or more become candidates. The SVM keeps non-background boxes with confidence of at least 0.60, and non-max suppression at IoU 0.3 removes duplicates.

3 Results and Discussion

3.1 Classification Accuracy on the Test Set

Classifier	Best setting	Validation accuracy	Test accuracy
k-NN	k = 5, distance weights	85.7%	68.7%
RBF-SVM	C = 5, gamma = scale	92.3%	87.2%

Table 2: Validation and test accuracy of the two classifiers

The SVM beats k-NN by about 18.5 points on test data. It loses 5 points from validation to test while k-NN loses 17, likely because validation folds hold augmented copies of training crops. The SVM macro F1 is 0.848 and 20 of 43 classes reach F1 of 0.90 or more.

3.2 Per-Class Performance of the SVM

Strongest classes (F1)	Weakest classes (F1)
Yield · 0.997	Pedestrians · 0.342
Go straight or right · 0.996	Beware ice/snow · 0.510
Vehicles >3.5t prohibited · 0.980	Speed limit 20 · 0.583
No entry · 0.979	Road narrows right · 0.623
Priority road · 0.978	Traffic signals · 0.642

Table 3: Five strongest and five weakest classes by F1-score

Distinct shapes (Yield, No entry) score above 0.97, while speed-limit discs (ids 0–8) and warning triangles (ids 18–31) are confused most.

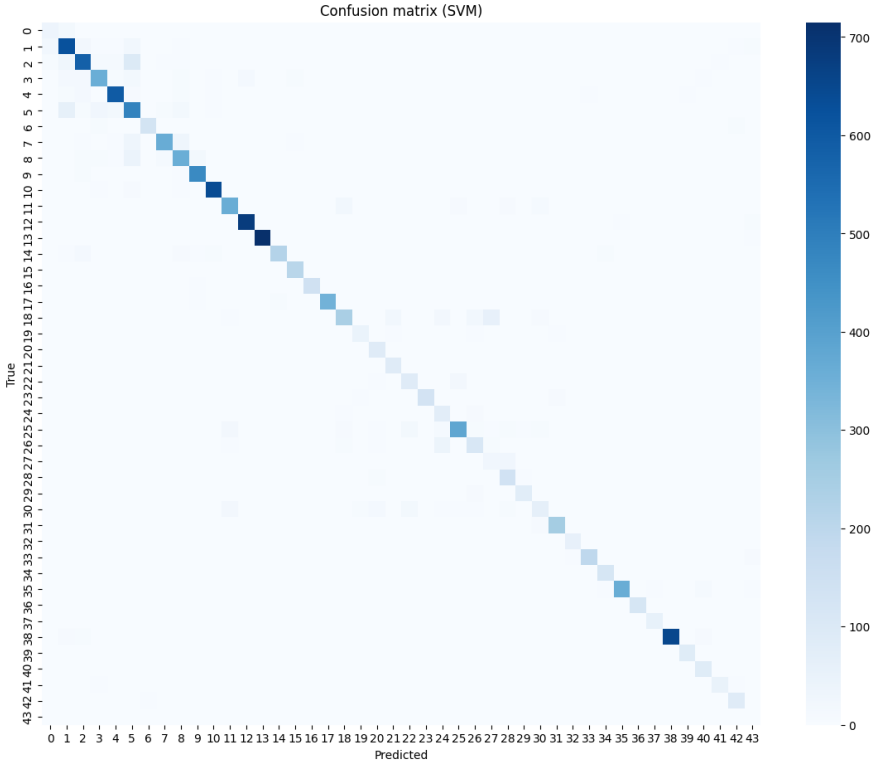


Figure 6: Confusion matrix of the SVM on the 12,630 test images (rows: true class, columns: predicted class 0–42 signs, 43 background)

3.3 Detection Results

Synthetic scene: Ground truth: Speed limit 100, Speed limit 30, General caution, Keep right. Predicted: Speed limit 100 (99%) and Speed limit 30 (92%), so 2 of 4 signs found with no false alarms.

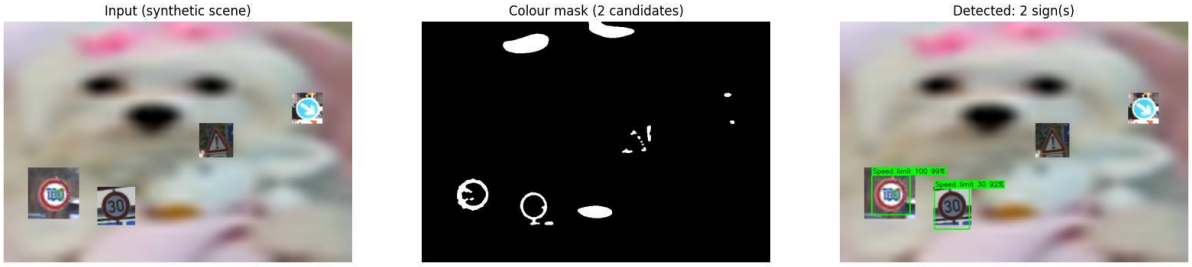


Figure 7: Synthetic scene: input, colour mask (2 candidates) and detections

Real photograph (speed20l.webp): Expected: Speed limit 20. Predicted: Speed limit 20 at 99% confidence.

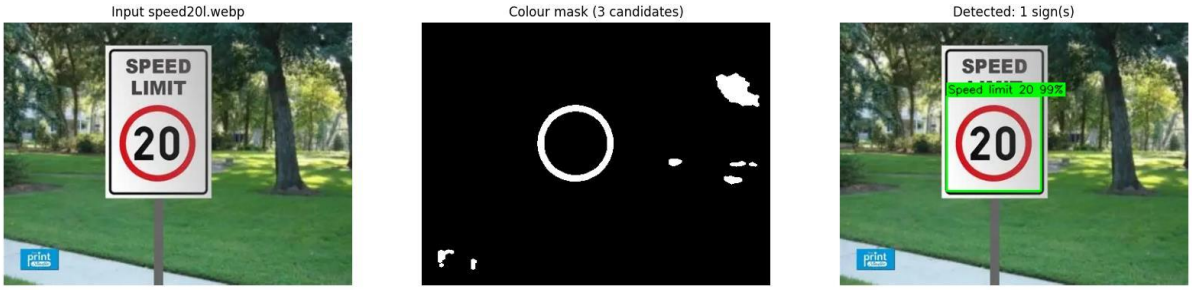


Figure 8: Real photograph: input, colour mask (3 candidates) and detection

3.4 Discussion

Both misses in the synthetic scene came from the candidate stage (colour mask and shape filters), not from the classifier.

3.5 Conclusion

This project successfully demonstrates a classical machine learning pipeline for traffic sign recognition. HOG shape features and HSV colour histograms with an RBF-SVM reach 87.2% test accuracy over 43 classes on 12,630 unseen GTSRB images, and the final model is trained in 118 s on a CPU. The SVM beats k-NN by about 18.5 points, while k-NN falls from 85.7% on validation to 68.7% on test.

Signs with distinct shapes, such as Yield and No entry, score an F1 above 0.97, while small digits, pedestrians and ice/snow signs are the hardest. The limits are that 32×32 crops and hand-crafted features lose detail, and that the colour mask misses low-contrast signs.

Overall, a classical HOG, colour and SVM pipeline is a fast, explainable TSR baseline, and deep detectors are the natural next step.

3.6 Future Scope

The current system provides a fast and explainable solution; however, there are several areas where it can be further enhanced to improve performance and applicability.

- The model can be **trained on all 26,640 images** instead of the 8,330-crop subset.
- A **small CNN** can be added as the classifier; ResNet reaches 99.7% on GTSRB [6].
- The colour mask can be replaced with a **learned detector** such as YOLO [3]–[5].
- The system can be tested on **full-scene data** and on **embedded hardware** such as Raspberry Pi or Jetson Nano [2].

References

[1] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, "Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition," Neural Networks, vol. 32, pp. 323–332, 2012, doi: 10.1016/j.neunet.2012.02.016.
Reference link: https://doi.org/10.1016/j.neunet.2012.02.016

[2] N. Triki, M. Karray, and M. Ksantini, "A comprehensive survey and analysis of traffic sign recognition systems with hardware implementation," IEEE Access, vol. 12, pp. 144069–144081, 2024, doi: 10.1109/ACCESS.2024.3459708.
Reference link: https://doi.org/10.1109/access.2024.3459708

[3] B. Meng and W. Shi, "Small traffic sign recognition method based on improved YOLOv7," Scientific Reports, vol. 15, 2025, doi: 10.1038/s41598-025-88679-w.
Reference link: https://doi.org/10.1038/s41598-025-88679-w

[4] H. Zhang, M. Liang, and Y. Wang, "YOLO-BS: a traffic sign detection algorithm based on YOLOv8," Scientific Reports, vol. 15, 2025, doi: 10.1038/s41598-025-88184-0.
Reference link: https://doi.org/10.1038/s41598-025-88184-0

[5] C. Wang, B. Zheng, and C. Li, "Efficient traffic sign recognition using YOLO for intelligent transport systems," Scientific Reports, vol. 15, 2025, doi: 10.1038/s41598-025-98111-y.
Reference link: https://doi.org/10.1038/s41598-025-98111-y

[6] L. Wang, "Comparative experiment and performance evaluation of three different models of traffic sign recognition system," ITM Web of Conferences, vol. 78 (CSEIT 2025), Art. no. 03011, 2025, doi: 10.1051/itmconf/20257803011.
Reference link: https://doi.org/10.1051/itmconf/20257803011

[7] Dataset Source: "tanganke/gtsrb (German Traffic Sign Recognition Benchmark)," Hugging Face Datasets, 2026.

Reference link: https://huggingface.co/datasets/tanganke/gtsrb

[8] Dataset Source: "uoft-cs/cifar10," Hugging Face Datasets, 2026.
Reference link: https://huggingface.co/datasets/uoft-cs/cifar10

Appendix A: Python Code

The complete implementation was developed in Google Colab using Python, OpenCV and scikit-learn. The code is listed cell by cell in the order in which it is run.

Cell 1: Libraries and configuration

```
import cv2, time, joblib
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
IMG_SIZE      = 32
MAX_PER_CLASS = 200
AUG_COPIES    = 1
N_BACKGROUND  = 1500
CONF_THRESH   = 0.60
RANDOM_STATE  = 42
rng = np.random.RandomState(RANDOM_STATE)
```

```
CLASS_NAMES = [
    'Speed limit 20', 'Speed limit 30', 'Speed limit 50', 'Speed limit 60',
    # ... the 43 GTSRB sign names (ids 0-42) ...
    'End of no passing (>3.5t)', 'BACKGROUND']
BG = 43
```

Cell 2: Loading GTSRB and CIFAR-10

```
from datasets import load_dataset
```

```
class DS:
    """Makes a Hugging Face dataset behave like the torchvision one."""
    def __init__(self, hf, img_col, lbl_col='label'):
        self.hf, self.img_col = hf, img_col
        self.labels = np.array(hf[lbl_col])
    def __len__(self):
        return len(self.hf)
    def __getitem__(self, i):
        i = int(i)
        return self.hf[i][self.img_col], int(self.labels[i])
```

```
train_ds = DS(load_dataset('tanganke/gtsrb', split='train'), 'image')
test_ds  = DS(load_dataset('tanganke/gtsrb', split='test'),  'image')
cifar    = DS(load_dataset('uoft-cs/cifar10', split='train'), 'img')
print(len(train_ds), len(test_ds), len(cifar))
```

Cell 3: Sampling sign crops and test images

```
def pil_to_bgr(p):
    return cv2.cvtColor(np.array(p.convert('RGB')), cv2.COLOR_RGB2BGR)
```

```
labels_all = train_ds.labels
idx = []
for c in range(43):
    c_idx = np.where(labels_all == c)[0]
    if MAX_PER_CLASS and len(c_idx) > MAX_PER_CLASS:
        c_idx = rng.choice(c_idx, MAX_PER_CLASS, replace=False)
    idx.extend(c_idx)
sign_imgs   = [pil_to_bgr(train_ds[i][0]) for i in idx]
sign_labels = list(labels_all[idx])
```

```
X_test_raw = [pil_to_bgr(test_ds[i][0]) for i in range(len(test_ds))]
y_test     = test_ds.labels
print(len(sign_imgs), 'train |', len(X_test_raw), 'test')
```

Cell 4: Augmentation function and preview

```
def augment(img):
    h, w = img.shape[:2]
    M = cv2.getRotationMatrix2D((w / 2, h / 2), rng.uniform(-15, 15),
                                rng.uniform(0.85, 1.15))
    M[:, 2] += rng.uniform(-0.08, 0.08, 2) * [w, h]
    out = cv2.warpAffine(img, M, (w, h), borderMode=cv2.BORDER_REFLECT)
    out = cv2.convertScaleAbs(out, alpha=rng.uniform(0.5, 1.5),
                              beta=rng.uniform(-50, 50))
    r = rng.rand()
    if r < 0.25:
        out = cv2.GaussianBlur(out, (5, 5), 0)
```

```
    elif r < 0.45:
        k = np.zeros((5, 5), np.float32); k[2, :] = 1 / 5
        out = cv2.filter2D(out, -1, k)
    if rng.rand() < 0.4:
        out = np.clip(out + rng.normal(0, 8, out.shape), 0, 255).astype(np.uint8)
    return out

fig, axes = plt.subplots(2, 8, figsize=(16, 4.5))
for k in range(8):
    i = rng.randint(len(sign_imgs))
    axes[0, k].imshow(cv2.cvtColor(cv2.resize(sign_imgs[i], (64, 64)),
                                   cv2.COLOR_BGR2RGB))
    axes[1, k].imshow(cv2.cvtColor(cv2.resize(augment(sign_imgs[i]), (64, 64)),
                                   cv2.COLOR_BGR2RGB))
    axes[0, k].axis('off'); axes[1, k].axis('off')
plt.show()
```

Cell 5: Background class and training set

```
bg_imgs = []
for i in rng.choice(len(cifar), N_BACKGROUND, replace=False):
    im = pil_to_bgr(cifar[int(i)][0])
    if rng.rand() < 0.5:
        s = rng.randint(12, 28); x, y = rng.randint(0, 32 - s, 2)
        im = im[y:y + s, x:x + s]
    bg_imgs.append(cv2.resize(im, (48, 48)))
for _ in range(300):
    base = rng.randint(0, 255, 3).astype(np.uint8)
    patch = np.tile(base, (48, 48, 1)) + rng.normal(0, 6, (48, 48, 3))
    bg_imgs.append(np.clip(patch, 0, 255).astype(np.uint8))

train_imgs, train_labels = list(sign_imgs), list(sign_labels)
for im, lb in zip(sign_imgs, sign_labels):
    for _ in range(AUG_COPIES):
        train_imgs.append(augment(im)); train_labels.append(lb)
train_imgs += bg_imgs; train_labels += [BG] * len(bg_imgs)
y_train = np.array(train_labels)
print('Total training samples:', len(train_imgs))
```

Cell 6: Pre-processing and feature extraction

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(4, 4))

def preprocess(img_bgr):
    img = cv2.resize(img_bgr, (IMG_SIZE, IMG_SIZE), interpolation=cv2.INTER_AREA)
    img = cv2.GaussianBlur(img, (3, 3), 0)
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
    lab[:, :, 0] = clahe.apply(lab[:, :, 0])
    return cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)

hog = cv2.HOGDescriptor(_winSize=(IMG_SIZE, IMG_SIZE), _blockSize=(16, 16),
                        _blockStride=(8, 8), _cellSize=(8, 8), _nbins=9)

def extract_features(img_bgr):
    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
    hog_feat = hog.compute(gray).ravel()
    hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
    h = cv2.calcHist([hsv], [0], None, [16], [0, 180])
    s = cv2.calcHist([hsv], [1], None, [8], [0, 256])
    v = cv2.calcHist([hsv], [2], None, [8], [0, 256])
    col = np.concatenate([h, s, v]).ravel()
    col /= (col.sum() + 1e-7)
    return np.concatenate([hog_feat, col]).astype(np.float32)

feat = lambda im: extract_features(preprocess(im))

t0 = time.time()
X_train = np.array([feat(im) for im in train_imgs])
X_test  = np.array([feat(im) for im in X_test_raw])
print(X_train.shape, X_test.shape, '%.1fs' % (time.time() - t0))
```

Cell 7: Scaling, PCA and grid search

```
X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train, test_size=0.2,
                                            stratify=y_train,
                                            random_state=RANDOM_STATE)

def make_pipe(clf):
    return Pipeline([('scaler', StandardScaler()),
                     ('pca', PCA(n_components=0.95, random_state=RANDOM_STATE)),
                     ('clf', clf)])

knn = GridSearchCV(make_pipe(KNeighborsClassifier()),
                   {'clf__n_neighbors': [1, 3, 5],
                    'clf__weights': ['uniform', 'distance']},
                   cv=3, n_jobs=-1).fit(X_tr, y_tr)
print('k-NN best:', knn.best_params_, '| val acc %.4f' % knn.score(X_val, y_val))

t0 = time.time()
svm = GridSearchCV(make_pipe(SVC(kernel='rbf', gamma='scale')),
```

```
                  {'clf__C': [5, 20]}, cv=3, n_jobs=-1).fit(X_tr, y_tr)
print('SVM best:', svm.best_params_, '| val acc %.4f' % svm.score(X_val, y_val),
      '| %.0fs' % (time.time() - t0))
```

Cell 8: Final SVM training and saving

```
t0 = time.time()
best_C = svm.best_params_['clf__C']
model = make_pipe(SVC(kernel='rbf', C=best_C, gamma='scale', probability=True,
                      random_state=RANDOM_STATE)).fit(X_train, y_train)
print('Trained in %.0fs' % (time.time() - t0))
joblib.dump(model, 'traffic_sign_svm.pkl')
```

Cell 9: Test evaluation

```
pred_knn = knn.best_estimator_.predict(X_test)
pred_svm = model.predict(X_test)
print('k-NN test accuracy: %.4f' % accuracy_score(y_test, pred_knn))
print('SVM  test accuracy: %.4f' % accuracy_score(y_test, pred_svm))

plt.bar(['k-NN', 'SVM'], [accuracy_score(y_test, pred_knn) * 100,
                          accuracy_score(y_test, pred_svm) * 100],
        color=['#4C72B0', '#DD8452']); plt.ylim(0, 100)
plt.ylabel('Test accuracy (%)'); plt.show()

print(classification_report(y_test, pred_svm, labels=list(range(43)),
                            target_names=CLASS_NAMES[:43], digits=3,
                            zero_division=0))

cm = confusion_matrix(y_test, pred_svm, labels=list(range(44)))
plt.figure(figsize=(14, 11)); sns.heatmap(cm, cmap='Blues')
plt.xlabel('Predicted'); plt.ylabel('True')
plt.title('Confusion matrix (SVM)'); plt.show()
```

Cell 10: Colour mask, candidate boxes and NMS

```
def colour_mask(bgr):
    hsv = cv2.cvtColor(cv2.GaussianBlur(bgr, (5, 5), 0), cv2.COLOR_BGR2HSV)
    red1 = cv2.inRange(hsv, (0, 90, 60),   (10, 255, 255))
    red2 = cv2.inRange(hsv, (165, 90, 60), (180, 255, 255))
    blue = cv2.inRange(hsv, (100, 120, 50), (130, 255, 255))
    yell = cv2.inRange(hsv, (15, 110, 120), (35, 255, 255))
    mask = red1 | red2 | blue | yell
    k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, k, iterations=2)
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, k, iterations=1)
    return mask

def find_candidates(bgr, min_frac=0.0004, max_frac=0.25):
    H, W = bgr.shape[:2]
    mask = colour_mask(bgr)
    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    boxes = []
    for c in cnts:
        x, y, w, h = cv2.boundingRect(c)
        area_frac = (w * h) / float(W * H)
        aspect = w / float(h)
        fill = cv2.contourArea(c) / float(w * h)
        if (min_frac < area_frac < max_frac and 0.6 < aspect < 1.5
                and fill > 0.30 and min(w, h) >= 16):
            pad = int(0.12 * max(w, h))
            x0, y0 = max(0, x - pad), max(0, y - pad)
            x1, y1 = min(W, x + w + pad), min(H, y + h + pad)
            boxes.append((x0, y0, x1 - x0, y1 - y0))
    return boxes, mask

def nms(dets, iou_thr=0.3):
    if not dets: return []
    b = [list(map(int, d[:4])) for d in dets]; s = [float(d[4]) for d in dets]
    keep = cv2.dnn.NMSBoxes(b, s, 0.0, iou_thr)
    return [dets[int(i)] for i in np.array(keep).flatten()]
```

Cell 11: Detection, drawing and display

```
def detect_and_classify(bgr, conf_thresh=CONF_THRESH, max_side=1000):
    h, w = bgr.shape[:2]
    if max(h, w) > max_side:
        f = max_side / float(max(h, w))
        bgr = cv2.resize(bgr, None, fx=f, fy=f, interpolation=cv2.INTER_AREA)
    boxes, mask = find_candidates(bgr)
    dets = []
    if boxes:
        feats = np.array([feat(bgr[y:y + h_, x:x + w_])
                          for x, y, w_, h_ in boxes])
        probs = model.predict_proba(feats)
        classes = model.classes_
        for (x, y, w_, h_), p in zip(boxes, probs):
            j = int(np.argmax(p)); label, score = int(classes[j]), float(p[j])
```

```
            if label != BG and score >= conf_thresh:
                dets.append((x, y, w_, h_, score, label))
    return nms(dets), bgr, mask, boxes

def draw(bgr, dets):
    out = bgr.copy()
    for x, y, w, h, s, lb in dets:
        cv2.rectangle(out, (x, y), (x + w, y + h), (0, 255, 0), 2)
        txt = '%s %.0f%%' % (CLASS_NAMES[lb], s * 100)
        (tw, th), _ = cv2.getTextSize(txt, cv2.FONT_HERSHEY_SIMPLEX, 0.5, 1)
        cv2.rectangle(out, (x, max(0, y - th - 6)), (x + tw + 4, y),
                      (0, 255, 0), -1)
        cv2.putText(out, txt, (x + 2, max(th, y - 4)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 1)
    return out

def show_result(bgr, title=''):
    dets, img, mask, boxes = detect_and_classify(bgr)
    fig, ax = plt.subplots(1, 3, figsize=(20, 6))
    ax[0].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    ax[0].set_title('Input ' + title)
    ax[1].imshow(mask, cmap='gray')
    ax[1].set_title('Colour mask (%d candidates)' % len(boxes))
    ax[2].imshow(cv2.cvtColor(draw(img, dets), cv2.COLOR_BGR2RGB))
    ax[2].set_title('Detected: %d sign(s)' % len(dets))
    for a in ax: a.axis('off')
    plt.show()
    for d in dets: print('  ->', CLASS_NAMES[d[5]], '(%.0f%%)' % (d[4] * 100))
    return dets
```

Cell 12: Synthetic demo scene

```
def make_demo_scene(n_signs=4, W=800, H=550):
    bg = pil_to_bgr(cifar[int(rng.randint(len(cifar)))][0])
    scene = cv2.GaussianBlur(cv2.resize(bg, (W, H),
                             interpolation=cv2.INTER_CUBIC), (15, 15), 0)
    truth, slot = [], W // n_signs
    for k in range(n_signs):
        i = int(rng.randint(len(test_ds)))
        size = int(rng.randint(70, 120))
        sign = cv2.resize(X_test_raw[i], (size, size),
                          interpolation=cv2.INTER_CUBIC)
        sign = cv2.convertScaleAbs(sign, alpha=rng.uniform(0.7, 1.3),
                                   beta=rng.uniform(-25, 25))
        x = k * slot + int(rng.randint(10, max(11, slot - size - 10)))
        y = int(rng.randint(40, H - size - 40))
        scene[y:y + size, x:x + size] = sign
        truth.append(CLASS_NAMES[y_test[i]])
    return scene, truth

scene, truth = make_demo_scene()
print('Ground truth:', truth)
_ = show_result(scene, '(synthetic scene)')
```

Cell 13: Real photograph demo

```
from google.colab import files
uploaded = files.upload()
for name in uploaded:
    im = cv2.imread(name)
    print('\n==', name, im.shape)
    show_result(im, name)
```